

PetCare Hub

Documentacao Tecnica do Banco de Dados

Challenge FIAP 2026 — CLYVO VET

Sprint 3 — Mastering Relational and Non-Relational Database

Integrantes

Nome	RM
Alexander Dennis Isidro Mamani	565554
Kelson Zhang	563748

1. Explicação da Modelagem

O banco de dados do projeto **PetCare Hub** foi modelado para apoiar a continuidade do cuidado veterinário dentro do contexto do Challenge FIAP 2026 — CLYVO VET. A estrutura foi desenvolvida com foco em organização, integridade dos dados, escalabilidade e integração com as APIs Java Spring Boot e .NET do projeto.

O objetivo principal da modelagem é centralizar informações clínicas, preventivas e operacionais do ecossistema PetCare Hub, permitindo o gerenciamento completo da jornada de saúde do pet. O banco foi projetado para armazenar dados relacionados aos tutores, clínicas veterinárias, pets, consultas, protocolos preventivos, dispositivos IoT, leituras de sensores, alertas de saúde e score de saúde.

A modelagem foi construída seguindo os princípios da modelagem relacional e da 3ª Forma Normal (3FN), reduzindo redundâncias e garantindo consistência dos dados. Além disso, a estrutura foi organizada para permitir consultas analíticas, geração de relatórios, monitoramento contínuo do pet e integração com futuras funcionalidades da plataforma.

O projeto também foi pensado para suportar funcionalidades preventivas e inteligentes, utilizando dados clínicos e leituras de sensores para geração de alertas e acompanhamento longitudinal da saúde animal. Dessa forma, o banco de dados atua como base central para as regras de negócio, APIs, dashboards clínicos e futuras evoluções do ecossistema PetCare Hub.

O banco possui 14 tabelas: **TUTOR**, **CLINICA**, **PET**, **CONSULTA**, **PROTOCOLO_PREVENTIVO**, **EVENTO_PREVENTIVO**, **DISPOSITIVO_IOT**, **LEITURA_COLEIRA**, **LEITURA_COMEDOURO**, **LEITURA_AMBIENTE**, **ALERTA_SAUDE**, **SCORE_SAUDE**, **LOG_ERROS** e **AUDITORIA_TUTOR** (criada pelo trigger de auditoria). A tabela TUTOR nasceu como **RESPONSAVEL** e foi renomeada para alinhar com a terminologia usada no restante do produto.

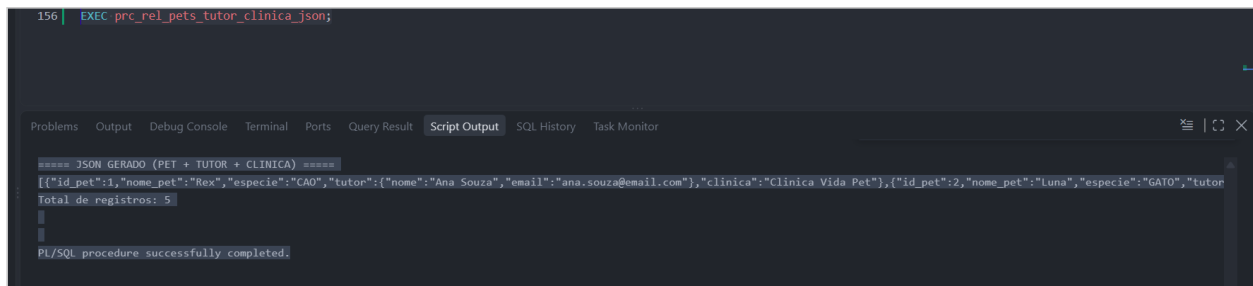
2. Desenvolvimento de 02 Procedures

As duas procedures abaixo atendem ao requisito da rubrica oficial da Sprint 3: cada uma trata pelo menos 3 exceções específicas (além de OTHERS). A seguir, prints reais da execução no Oracle SQL Developer, com um caso de sucesso para cada uma e a evidência das exceções registradas na tabela LOG_ERROS.

2.1 PRC_REL_PETS_TUTOR_CLINICA_JSON — JOIN + conversão para JSON manual

Faz JOIN entre PET, TUTOR e CLINICA e monta um array JSON linha a linha, chamando a function FN_PET_PARA_JSON (sem usar TO_JSON/JSON_OBJECT). Trata 4 exceções distintas.

Caso de sucesso (execução real — 5 pets):



```
156 | EXEC prc_rel_pets_tutor_clinica_json;

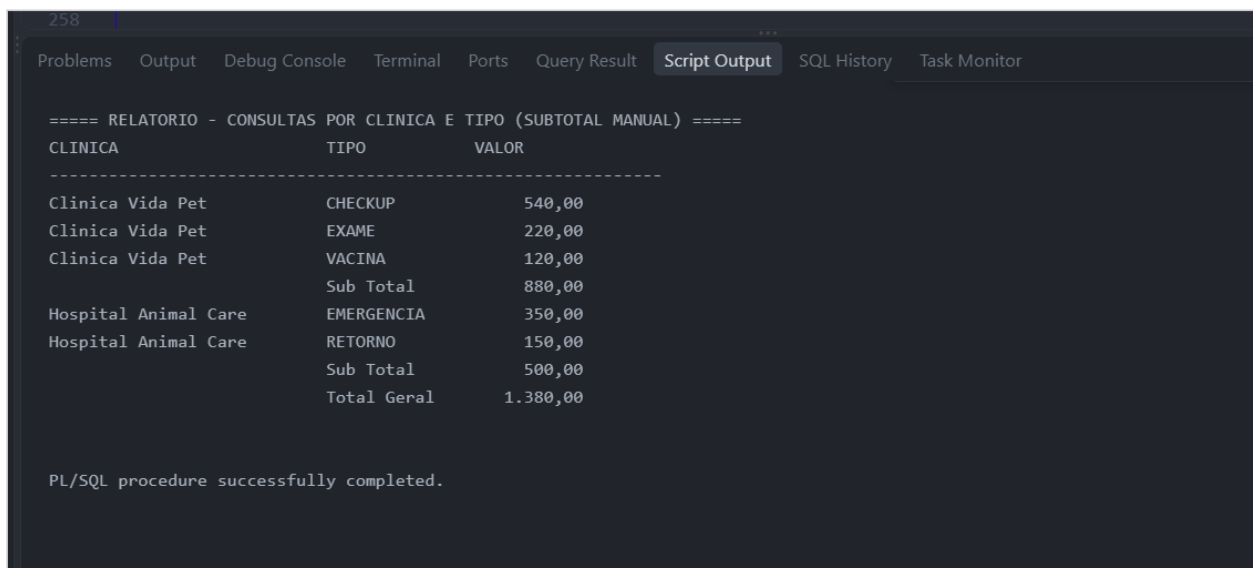
==== JSON GERADO (PET + TUTOR + CLINICA) ====
[{"id_pet":1,"nome_pet":"Rex","especie":"CAO","tutor":{"nome":"Ana Souza","email":"ana.souza@email.com"},"clinica":"Clinica Vida Pet"}, {"id_pet":2,"nome_pet":"Luna","especie":"GATO","tutor":
Total de registros: 5

PL/SQL procedure successfully completed.
```

2.2 PRC_REL_CONSULTAS_SUBTOTAL — subtotal e total geral manuais

Lê CONSULTA (fato) categorizada por CLINICA — categoria 1, identificada por id_clinica para não misturar clínicas homônimas — e TIPO_CONSULTA (categoria 2), com o cursor trazendo as linhas BRUTAS (uma por consulta, sem SUM, GROUP BY, ROLLUP, CUBE ou GROUPING SETS). Tudo é acumulado MANUALMENTE em variáveis PL/SQL dentro do próprio loop, em dois níveis: (1) os valores somados por combinação completa das categorias (clínica, tipo) — várias consultas da mesma clínica+tipo viram uma única linha com o valor somado; (2) um subtotal por grupo da categoria 1 (clínica); e um total geral ao final. O layout segue o modelo oficial da rubrica (relatório Agência/Conta/Saldo): a categoria 1 se repete em toda linha de combinação, e os valores de agrupamento ficam ausentes (em branco) somente nas linhas de "Sub Total" e "Total Geral". Trata 4 exceções distintas.

Caso de sucesso (execução real):



```
258 |

==== RELATORIO - CONSULTAS POR CLINICA E TIPO (SUBTOTAL MANUAL) ====
CLINICA          TIPO          VALOR
-----
Clinica Vida Pet  CHECKUP       540,00
Clinica Vida Pet  EXAME        220,00
Clinica Vida Pet  VACINA       120,00
                  Sub Total    880,00
Hospital Animal Care EMERGENCIA    350,00
Hospital Animal Care RETORNO       150,00
                  Sub Total    500,00
                  Total Geral 1.380,00

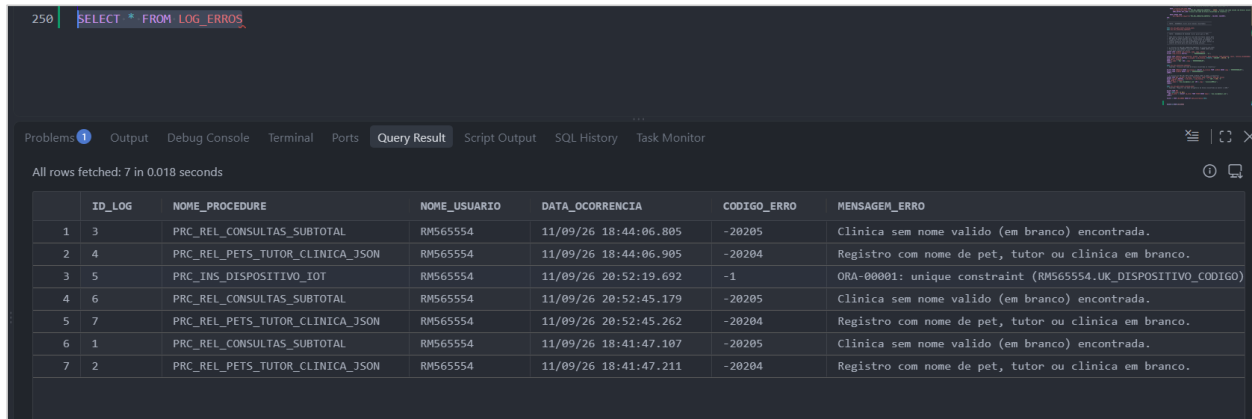
PL/SQL procedure successfully completed.
```

Prova da soma manual por combinação completa: a Clinica Vida Pet tem 3 consultas do tipo CHECKUP (180,00 + 170,00 + 190,00), que a procedure acumula e exibe como uma única linha de 540,00 — em vez de 3 linhas

separadas — antes de somar com EXAME (220,00) e VACINA (120,00) no Sub Total da clínica (880,00), confirmando os dois níveis de acumulação manual (combinação clínica+tipo e subtotal por clínica) exigidos pela rubrica.

2.3 Evidência das exceções — tabela LOG_ERROS

Print real de `SELECT * FROM LOG_ERROS ORDER BY data_ocorrencia DESC;` — confirma que as exceções de nome em branco das duas procedures (e_clinica_sem_nome, e_dados_incompletos) foram registradas oficialmente na tabela de log, em duas rodadas de teste distintas.



	ID_LOG	NOME_PROCEDURE	NOME_USUARIO	DATA_OCORRENCIA	CODIGO_ERRO	MENSAGEM_ERRO
1	3	PRC_REL_CONSULTAS_SUBTOTAL	RM565554	11/09/26 18:44:06.805	-20205	Clinica sem nome valido (em branco) encontrada.
2	4	PRC_REL_PETS_TUTOR_CLINICA_JSON	RM565554	11/09/26 18:44:06.905	-20204	Registro com nome de pet, tutor ou clinica em branco.
3	5	PRC_INS_DISPOSITIVO_IOT	RM565554	11/09/26 20:52:19.692	-1	ORA-00001: unique constraint (RM565554.UK_DISPPOSITIVO_CODIGO)
4	6	PRC_REL_CONSULTAS_SUBTOTAL	RM565554	11/09/26 20:52:45.179	-20205	Clinica sem nome valido (em branco) encontrada.
5	7	PRC_REL_PETS_TUTOR_CLINICA_JSON	RM565554	11/09/26 20:52:45.262	-20204	Registro com nome de pet, tutor ou clinica em branco.
6	1	PRC_REL_CONSULTAS_SUBTOTAL	RM565554	11/09/26 18:41:47.107	-20205	Clinica sem nome valido (em branco) encontrada.
7	2	PRC_REL_PETS_TUTOR_CLINICA_JSON	RM565554	11/09/26 18:41:47.211	-20204	Registro com nome de pet, tutor ou clinica em branco.

3. Desenvolvimento de 02 Functions

Além das duas functions de negócio do produto (FN_CALCULAR_IDADE_PET e FN_SCORE_MEDIO_PET, compiladas com sucesso), a rubrica da Sprint 3 exige 2 functions com regras próprias: conversão manual para JSON e validação de senha — ambas com tratamento de exceções nomeadas, demonstradas a seguir com prints reais da execução.

3.1 FN_PET_PARA_JSON — conversão relacional → JSON manual

Monta manualmente (concatenação de string) um JSON a partir de PET + TUTOR + CLINICA. Não usa TO_JSON, JSON_OBJECT, JSON_VALUE nem nenhuma função built-in de JSON do Oracle. Escapa manualmente aspas, barra invertida, quebra de linha e tabulação. Trata 4 exceções distintas.

Caso de sucesso:

```
125 SELECT fn_pet_para_json(  
126     p.id_pet, p.nome, p.especie, t.nome, t.email, c.nome  
127 ) AS json_pet  
128 FROM PET p  
129 JOIN TUTOR t ON t.id_tutor = p.id_tutor  
130 JOIN CLINICA c ON c.id_clinica = p.id_clinica  
131 WHERE p.nome = 'Rex';  
132  
133 -- Caso de excecao da Funcao 1 (id pet nulo)  
134 SELECT fn_pet_para_json(NULL, 'Teste', 'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste') AS json_erro  
135 FROM DUAL;  
136  
137 -- Caso de escape manual (barra invertida, aspas, quebra de  
138 -- linha e tabulacao no mesmo valor) - print para a documentacao  
139 SELECT fn_pet_para_json(  
140     99,  
141     'Nome com "aspas" e barra \ contra' || CHR(10) || 'quebra de linha' || CHR(9) || 'tab',  
142     'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste'  
143 ) AS json_escapa_teste  
144 FROM DUAL;
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.030 seconds

	JSON_PET
1	{"id_pet":1,"nome_pet":"Rex","especie":"CAO","tutor":{"nome":"Ana Souza","email":"ana.souza@email.com"},"clinica":"Clinica Vida Pet"}

Caso de exceção (id_pet nulo):

```
SELECT fn_pet_para_json(NULL, 'Teste', 'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste') AS json_erro  
FROM DUAL;  
  
-- Caso de escape manual (barra invertida, aspas, quebra de  
-- linha e tabulacao no mesmo valor) - print para a documentacao  
SELECT fn_pet_para_json(  
    99,  
    'Nome com "aspas" e barra \ contra' || CHR(10) || 'quebra de linha' || CHR(9) || 'tab',  
    'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste'  
)
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.014 seconds

	JSON_ERRO
1	{"erro":"id_pet obrigatorio para gerar JSON"}

Caso de escape manual (aspas, barra invertida, quebra de linha e tabulação no mesmo valor):

```
139 SELECT fn_pet_para_json(  
140     99,  
141     'Nome com "aspas" e barra \ contra' || CHR(10) || 'quebra de linha' || CHR(9) || 'tab',  
142     'CAO', 'Tutor Teste', 'teste@teste.com', 'Clinica Teste'  
143 ) AS json_escapa_teste  
144 FROM DUAL;  
145  
146 -- Caso de sucesso da Funcao 2  
147 SELECT fn_validar_forca_senha('Senha123') AS resultado FROM DUAL;
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.015 seconds

	JSON_ESCAPE_TESTE
1	{"id_pet":99,"nome_pet":"Nome com \aspas\ e barra \ contra\ quebra de linha\ttab","especie":"CAO","tutor":{"nome":"Tutor Teste","email":"teste@teste.com"},"clinica":"Cli..."}

3.2 FN_VALIDAR_FORCA_SENHA — validação de senha do fluxo de ativação do TUTOR

Valida a senha usada na ativação de conta do TUTOR (mínimo 8 caracteres, com letra e número). Trata 4 exceções distintas.

Caso de sucesso ('Senha123'):

```
146 -- Caso de sucesso da Funcao 2
147 SELECT fn_validar_forca_senha('Senha123') AS resultado FROM DUAL;
148
149 -- Casos de execucao da Funcao 2 (print para a documentacao)
150 SELECT fn_validar_forca_senha(NULL) AS resultado FROM DUAL;
151 SELECT fn_validar_forca_senha('abc') AS resultado FROM DUAL;
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.073 seconds

	RESULTADO
1	VALIDA

Caso de exceção — senha nula:

```
150 SELECT fn_validar_forca_senha(NULL) AS resultado FROM DUAL;
151 SELECT fn_validar_forca_senha('abc') AS resultado FROM DUAL;
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.015 seconds

	RESULTADO
1	INVALIDA: senha nao informada.

Caso de exceção — senha curta ('abc'):

```
151 SELECT fn_validar_forca_senha('abc') AS resultado FROM DUAL;
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.016 seconds

	RESULTADO
1	INVALIDA: senha deve ter no minimo 8 caracteres.

Caso de exceção — senha sem número ('semnumero'):

```
152 SELECT fn_validar_forca_senha('semnumero') AS resultado FROM DUAL;
153
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 1 in 0.013 seconds

	RESULTADO
1	INVALIDA: senha deve conter ao menos um numero.

4. Desenvolvimento de 01 Trigger para Auditoria

O trigger TRG_AUDITORIA_TUTOR dispara AFTER INSERT OR UPDATE OR DELETE ON TUTOR, FOR EACH ROW, e grava em AUDITORIA_TUTOR a operação, o usuário do banco (USER), a data/hora (SYSTIMESTAMP) e o status_acesso/email antes e depois da mudança. A tabela AUDITORIA_TUTOR, a sequence SEQ_AUDITORIA_TUTOR e o trigger foram criados com sucesso, e a demonstração ponta a ponta abaixo executou um INSERT, um UPDATE (PRE_CADASTRADO → ATIVO) e um DELETE em um tutor de teste.

Print real de SELECT * FROM AUDITORIA_TUTOR (filtrado pelo tutor de teste):

```
96 SELECT id_auditoria, id_tutor, operacao, usuario_bd, data_hora,
97        status_anterior, status_novo, email_anterior, email_novo
98 FROM AUDITORIA_TUTOR
99 WHERE email_anterior = 'auditoria@teste.com' OR email_novo = 'auditoria@teste.com'
100 ORDER BY data_hora;
101 -- Esperado: 3 linhas (INSERT, UPDATE, DELETE) para esse tutor,
102 -- com status_anterior/status_novo mostrando PRE_CADASTRADO -> ATIVO
103 -- no UPDATE, e id_tutor/email preenchidos em todas as 3 linhas.
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 3 in 0.036 seconds

	ID_AUDITORIA	ID_TUTOR	OPERACAO	USUARIO_BD	DATA_HORA	STATUS_ANTERIOR	STATUS_NOVO	EMAIL_ANTERIOR	EMAIL_NOVO
1	1	7	INSERT	RM565554	11/09/26 18:47:54.489	(null)	PRE_CADASTRADO	(null)	auditoria@teste.com
2	2	7	UPDATE	RM565554	11/09/26 18:47:54.522	PRE_CADASTRADO	ATIVO	auditoria@teste.com	auditoria@teste.com
3	3	7	DELETE	RM565554	11/09/26 18:47:54.549	ATIVO	(null)	auditoria@teste.com	(null)

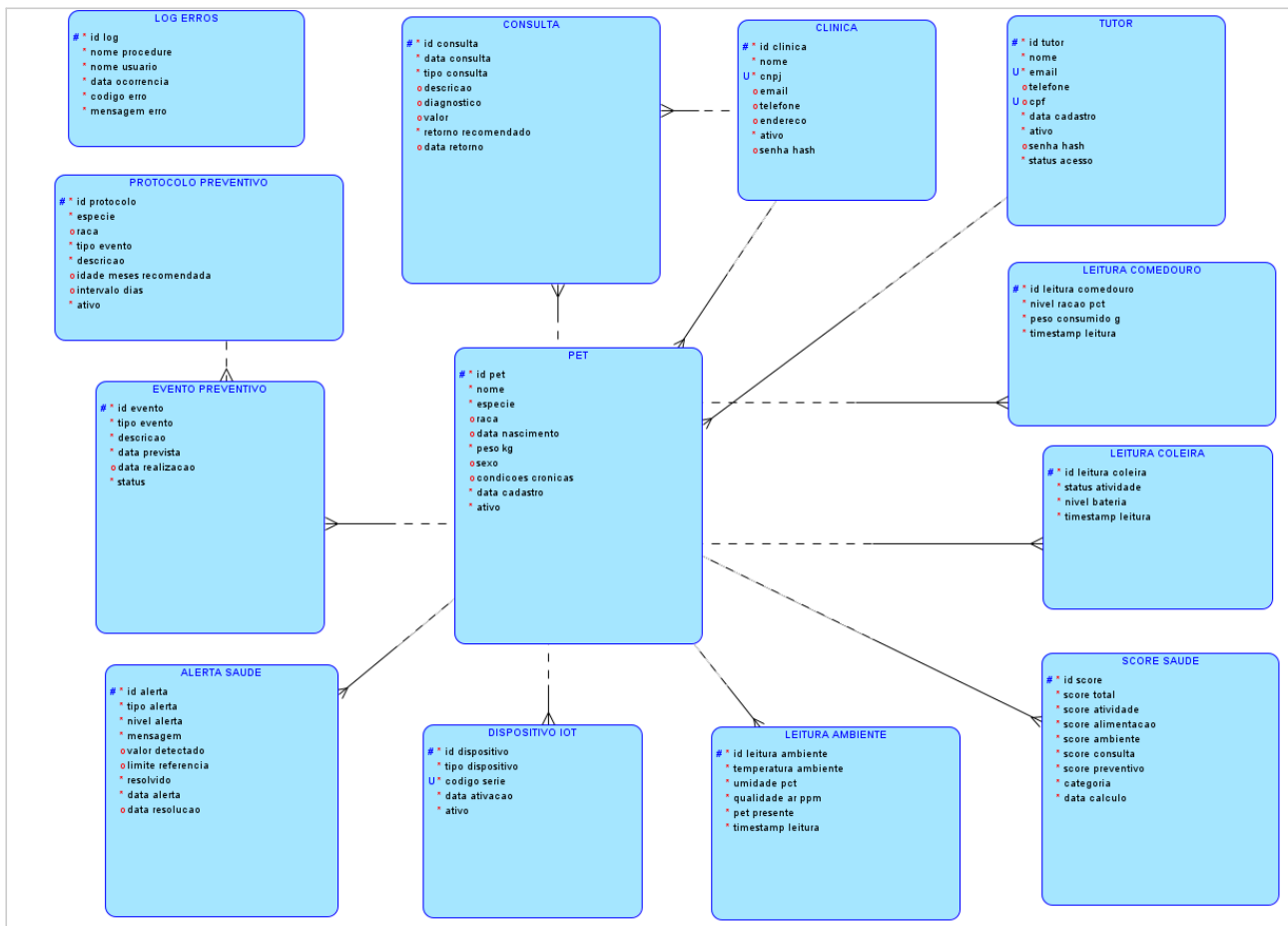
As 3 linhas confirmam a auditoria completa: quem alterou (usuário do banco), quando (data/hora com milissegundos) e o estado antes/depois de cada operação — com destaque para a transição PRE_CADASTRADO → ATIVO capturada no UPDATE.

5. Evidência de Carga Final de Dados

Print real da consulta de conferência final (UNION ALL sobre as 14 tabelas do banco), executada após a carga complementar. Confirma que todas as tabelas exigidas atingem o mínimo de 5 registros válidos.

	TABELA	TOTAL
1	TUTOR	5
2	CLINICA	5
3	PET	5
4	CONSULTA	7
5	PROTOCOLO_PREVENTIVO	6
6	EVENTO_PREVENTIVO	5
7	DISPOSITIVO_IOT	5
8	LEITURA_COLEIRA	7
9	LEITURA_COMEDOURO	8
10	LEITURA_AMBIENTE	9
11	ALERTA_SAUDE	7
12	SCORE_SAUDE	7
13	LOG_ERROS	7
14	AUDITORIA_TUTOR	3

6. DER



7. MER

